

RAMNIRANJAN JHUNJHUNWALA COLLEGE

(DEPARTMENT OF DSAI)

Ghatkopar (West), Mumbai – 86



Project Report

On

**Self-Healing Astronomer Airflow Pipeline
(AI-Powered Autonomous Failure Recovery System)**

In partial fulfillment of M.Sc. (DSAI)

By

Mr. Surajkumar Yadav

M.Sc. DSAI

University of Mumbai

(2025-2026)

RAMNIRANJAN JHUNJHUNWALA COLLEGE (AUTONOMOUS)



(Affiliated to university of Mumbai)

Ghatkopar (WEST), 400086

Certificate



University of Mumbai

*This is to certify that the project titled “**Self-Healing Astronomer Airflow Pipeline**” is a Bonafide work carried out by **Surajkumar Yadav** bearing Seat No:- **10404** submitted in partial fulfillment of the requirements for the degree of Master of Science in Data Science and Artificial Intelligence.*

(Signature of Internal Guide)

(Signature of Coordinator)

Examiner

Date:

College Seal

TABLE OF CONTENTS

Sr. No.	Title	Page No.
1	Acknowledgement	2
2	Abstract	4
3	Introduction	6
4	Literature Survey	8
5	Tools and Technologies used	12
6	System Architecture	17
7	Setting up Environment	25
8	Implementation/Working	29
9	Deployment	39
10	Limitation	45
11	Future Scope	48
12	Conclusion	51
13	References	53

Self-Healing Astronomer Airflow Pipeline

1. ACKNOWLEDGEMENT

1. ACKNOWLEDGEMENT

I would like to express my sincere and heartfelt gratitude to my professors and mentors for their continuous guidance, encouragement, and invaluable support throughout the course of this project. Their insightful suggestions, constructive feedback, and constant motivation played a crucial role in shaping this work and bringing it to completion.

I would also like to extend my appreciation to my peers and colleagues who supported me during the development and testing phases of this project. Their cooperation, discussions, and shared knowledge greatly contributed to the successful implementation of this system.

Finally, I would like to thank my family for their unwavering support, patience, and encouragement. Their belief in me has been a constant source of strength and inspiration throughout this journey.

2. ABSTRACT

2.ABSTRACT

Modern data pipelines are prone to failures due to issues such as code errors, schema mismatches, dependency failures, and infrastructure issues. These failures require manual debugging, which leads to increased downtime and operational overhead.

This project presents a Self-Healing Astro Airflow Pipeline that automatically detects, analyzes, and resolves failures using Artificial Intelligence.

The system captures failure logs from Airflow, processes them using an AWS Lambda-based event-driven architecture, and uses a Large Language Model (LLM) to identify root causes and generate code fixes.

A decision engine evaluates the safety of the generated fixes based on confidence and risk levels. Safe fixes are automatically committed to the repository through GitHub Pull Requests, while uncertain cases trigger structured email alerts.

The system supports multiple pipeline types, including file-based ingestion, analytics pipelines, and database ETL pipelines, demonstrating its scalability and adaptability.

This approach significantly reduces manual intervention and introduces a new paradigm of autonomous recovery in data engineering systems.

3. INTRODUCTION

3.INTRODUCTION

Data engineering pipelines are critical components of modern data-driven systems. These pipelines are responsible for ingesting, processing, and delivering data for analytics and machine learning applications.

However, pipeline failures are common due to various reasons such as:

- Code errors
- Data schema mismatches
- External system failures
- Configuration issues

Traditionally, these failures require manual debugging by engineers, which increases downtime and operational costs.

To address this problem, this project introduces a Self-Healing Airflow Pipeline, which leverages Artificial Intelligence to automatically detect, analyze, and fix failures.

The system integrates Apache Airflow, Astronomer, AWS Lambda, S3, and OpenAI's AI models to create an intelligent, automated recovery system.

The key objective of this project is to reduce manual intervention and enable autonomous recovery of data pipelines in a safe and controlled manner.

4. LITERATURE SURVEY

4.LITERATURE SURVEY

With the rapid growth of data-driven systems, data pipelines have become a critical component in modern organizations. These pipelines are responsible for extracting, transforming, and loading (ETL) data from multiple sources into storage systems for analytics and decision-making. However, maintaining the reliability of such pipelines remains a significant challenge due to frequent failures caused by code errors, data inconsistencies, infrastructure issues, and external system dependencies.

Traditional workflow orchestration tools such as Apache Airflow provide robust scheduling and monitoring capabilities. Airflow enables users to define workflows as Directed Acyclic Graphs (DAGs) and offers features such as task retries, logging, and alerting.

Basic Airflow Workflow:

[Data Source] → [Task 1] → [Task 2] → [Task 3] → [Output]

While these features help in detecting failures, they do not provide automated solutions for resolving them. In most cases, failures require manual intervention, where engineers analyze logs, identify root causes, and implement fixes. This process can be time-consuming and leads to increased system downtime.

Traditional Failure Handling:

[Failure Detected] → [Engineer Debugs] → [Fix Applied] → [Pipeline Restart]

To address these limitations, several monitoring and observability tools such as Prometheus, Grafana, and Datadog have been developed. These tools enhance visibility into system performance and provide real-time alerts. However, similar to Airflow, they primarily focus on detection and notification rather than automated recovery.

Monitoring System Flow:

[System Metrics] → [Monitoring Tool] → [Alert] → [Human Action]

Recent advancements in Artificial Intelligence, particularly Large Language Models (LLMs), have opened new possibilities in software engineering and automation. LLMs have demonstrated strong capabilities in understanding code, debugging errors, and generating solutions.

AI-Assisted Debugging:

[Error Logs] → [AI Model] → [Root Cause] → [Suggested Fix]

In parallel, the concept of self-healing systems has gained attention in distributed computing and cloud infrastructure. Self-healing systems are designed to automatically detect, diagnose, and recover from failures without human intervention.

Self-Healing System Concept:

[Failure] → [Detection] → [Diagnosis] → [Automatic Recovery]

However, most existing systems focus on infrastructure-level recovery (such as restarting services or scaling resources) rather than fixing application-level issues or code errors.

This project builds upon these existing concepts by combining workflow orchestration, cloud-based event-driven architecture, and AI-powered code understanding to create a fully autonomous self-healing pipeline.

Proposed System Enhancement:

[Failure] → [Log Analysis] → [AI Fix Generation] → [Decision Engine]

↓

[AUTO PR] or [EMAIL ALERT]

Unlike traditional systems that stop at failure detection, this system extends the pipeline lifecycle to include root cause analysis, fix generation, and safe deployment of solutions through GitHub Pull Requests.

Additionally, the system incorporates a decision engine that evaluates the reliability of AI-generated fixes using parameters such as confidence and risk level. This ensures that automation is applied in a controlled and safe manner.

By integrating Airflow, AWS services, and AI models, this project demonstrates a practical implementation of a self-healing data pipeline system. It highlights the transition from reactive monitoring systems to proactive and autonomous recovery systems, representing a significant advancement in the field of data engineering and intelligent systems.

5. TOOLS AND TECHNOLOGIES USED

1. TOOLS AND TECHNOLOGIES USED

1. Apache Airflow (Astronomer Runtime)

Apache Airflow is an open-source workflow orchestration platform used to design, schedule, and monitor data pipelines. In this project, Airflow is used to define workflows as Directed Acyclic Graphs (DAGs), where each task represents a specific operation in the pipeline.

The Astronomer Runtime provides a production-ready environment for Airflow with Kubernetes-based execution, ensuring scalability and reliability. A key feature utilized in this project is the failure callback mechanism, which captures task-level failures and initiates the self-healing process by storing logs in Amazon S3.

2. AWS Lambda

AWS Lambda is a serverless computing service that executes code in response to events without requiring server management. In this system, Lambda is triggered automatically when failure logs are uploaded to Amazon S3.

The Lambda function performs critical operations such as log parsing, error classification, context extraction, AI interaction, and decision-making. Based on the analysis, it either triggers an automated GitHub Pull Request or sends an email notification. Its serverless architecture ensures cost efficiency and scalability.

3. Amazon S3 (Simple Storage Service)

Amazon S3 is used as a storage layer for capturing Airflow failure logs and metadata. When a task fails, the system stores the traceback and contextual details in S3. S3 acts as a reliable and scalable intermediary that enables event-driven processing. The creation of objects in S3 automatically triggers the Lambda function, forming a key part of the system's architecture.

4. OpenAI API (Large Language Model)

The OpenAI API is used to integrate a Large Language Model capable of understanding code, analyzing logs, and generating fixes. The model processes structured input that includes error logs, DAG context, and source code. It produces a response containing the root cause, suggested fix, corrected code, confidence score, and risk level. This enables intelligent and automated debugging within the system.

5. GitHub

GitHub is used as the version control system for managing the project's codebase. The system interacts with GitHub using APIs to fetch source code files and create Pull Requests. When a fix is classified as safe, the system automatically creates a new branch, updates the file with corrected code, and raises a Pull Request. This ensures traceability, version control, and proper review workflows.

6. AWS Simple Email Service (SES)

AWS SES is used to send email notifications when automatic fixes are not applied. The system generates structured HTML emails containing detailed information such as the failure description, root cause, suggested fix, and confidence level. SES ensures reliable and scalable delivery of notifications, allowing users to take manual action when required.

7. Python Programming Language

Python is the primary programming language used throughout the project due to its simplicity, readability, and extensive library support. It is used for developing Airflow DAGs, AWS Lambda functions, and supporting modules. Python enables efficient handling of log parsing, API integration, data processing, and automation tasks.

8. Docker (for MySQL Setup)

Docker is used to create a containerized MySQL environment for testing database pipelines. It allows easy setup and isolation of the database without affecting the host system. This ensures a consistent and reproducible environment for development and testing.

9. MySQL Database

MySQL is used as a source database in one of the pipelines. It stores structured data that is extracted, transformed, and loaded into DynamoDB. This demonstrates the system's ability to handle traditional relational database workflows.

10. Amazon DynamoDB

Amazon DynamoDB is a NoSQL database used as the target storage system for processed data. It provides high performance, scalability, and low latency. It is used in multiple pipelines to store processed and transformed data efficiently.

11. Custom Logging and Processing Modules

Custom Python modules are developed for parsing logs, classifying errors, extracting task context, and building AI prompts. These modules form the core intelligence layer of the system. They ensure that raw logs are transformed into structured and meaningful input for the AI model.

12.GitHub REST API

The GitHub REST API is used to programmatically interact with repositories. It enables operations such as fetching file contents, creating branches, updating files, and generating Pull Requests. This integration is essential for enabling automated code modifications and maintaining proper version control practices.

The integration of these tools and technologies results in a robust, scalable, and intelligent system capable of autonomous failure detection, analysis, and recovery. The combination of orchestration, cloud services, and AI enables the system to move beyond traditional monitoring towards fully automated self-healing pipelines.

6. SYSTEM ARCHITECTURE

6.2 Event-Driven Workflow

The system is built on an event-driven model where each stage of processing is triggered automatically based on events.

The workflow begins when a task within an Airflow DAG fails during execution. Instead of relying solely on logs within Airflow, a custom failure callback is used to capture the error traceback and metadata. This information is then stored in Amazon S3, which acts as a persistent and scalable storage layer. The creation of a new log file in S3 triggers an AWS Lambda function, initiating the automated recovery process. This event-driven approach eliminates the need for continuous polling and ensures efficient resource utilization.

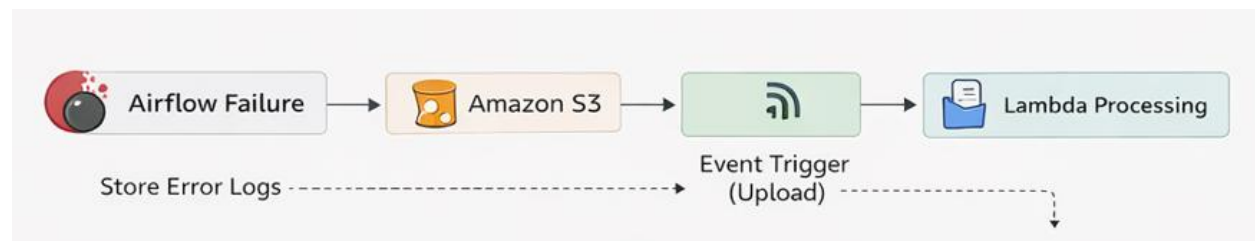


Fig 7.2: Event Driver Workflow

6.3 Failure Capture Mechanism

When a task fails in Airflow, a failure callback function is triggered. This function captures the complete traceback using Python's exception handling mechanism.

The captured information includes:

- Exception type
- Error message
- Full traceback
- DAG ID and Task ID
- Execution metadata

This information is stored in Amazon S3 in a structured format, ensuring that it can be easily processed by downstream components.

This approach is particularly important because certain Airflow environments, such as Astronomer Runtime, restrict direct access to logs. Therefore, capturing logs at the point of failure ensures compatibility across different execution environments.

6.4 Log Parsing and Error Classification

Once the Lambda function is triggered, it begins by parsing the log file stored in S3. The parsing module extracts relevant details such as exception type, message, and traceback.

The system then applies a classification algorithm to categorize the error into predefined types such as:

- Import errors
- Name errors
- Key errors
- Data schema errors
- Type or validation errors
- Connection and timeout errors

The classification process uses both rule-based logic and semantic analysis of error messages.

This ensures that even generic exceptions such as `ValueError` are correctly categorized based on their context.

The classification result includes:

- Error category
- Confidence score
- Recoverability flag

This structured classification is essential for guiding the decision engine in later stages.

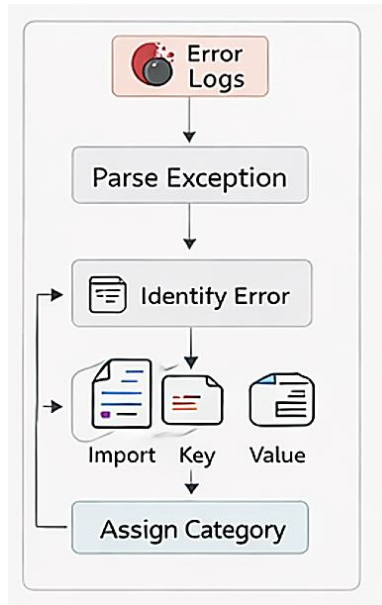


Fig 6.3 Error parsing and Error Classification

6.5 Context Extraction and Code Retrieval

After classification, the system builds contextual information required for AI-based debugging.

This includes:

- DAG information
- Failed task details
- Operator type
- Relevant imports
- Target file path extracted from traceback

The system then connects to the GitHub repository and retrieves the full source code of the affected file. Unlike traditional debugging approaches that rely on partial code snippets, this system provides the AI model with complete file context.

Providing full context improves the accuracy of AI-generated fixes and reduces ambiguity.

6.6 AI-Based Root Cause Analysis and Fix Generation

The extracted context is used to construct a structured prompt for the AI model. This prompt includes:

- Error logs
- Classification results
- Task code
- Full source file

The AI model processes this information and generates a response in a structured format containing:

- Root cause explanation
- Suggested fix
- Corrected code (full file)
- Confidence score
- Risk level

The requirement for full-file correction ensures that the generated fix is directly usable without manual modification.

6.7 Decision Engine

The decision engine is a critical component that determines whether an AI-generated fix should be applied automatically or sent for manual review.

The decision is based on:

- Confidence score
- Risk level
- Error category

Strict rules are implemented to ensure safe automation. For example:

- Only code-related fixes are eligible for automation
- High-confidence and low-risk fixes are prioritized
- Certain error types require stricter thresholds

This prevents unsafe or incorrect changes from being applied to the codebase.

6.8 Automated Code Update (GitHub Integration)

If the decision engine approves the fix, the system proceeds to create a Pull Request in GitHub.

The process includes:

- Creating a new branch
- Fetching the file SHA
- Replacing the file with corrected code
- Committing changes
- Creating a Pull Request

This ensures that all changes are tracked and reviewed before being merged into the main branch.

6.9 Notification System

If the fix is not deemed safe for automatic deployment, the system sends a detailed email notification using AWS SES.

The email includes:

- Failure details
- Root cause
- Suggested fix
- Corrected code

- Confidence level

This ensures that users are informed and can take appropriate action.

6.10 Scalability and Modularity

The architecture is designed to be scalable and modular.

- Serverless components (Lambda) ensure scalability
- S3 provides unlimited storage
- Airflow handles multiple pipelines
- Independent modules allow easy upgrades

This enables the system to handle multiple pipelines and failure scenarios simultaneously.

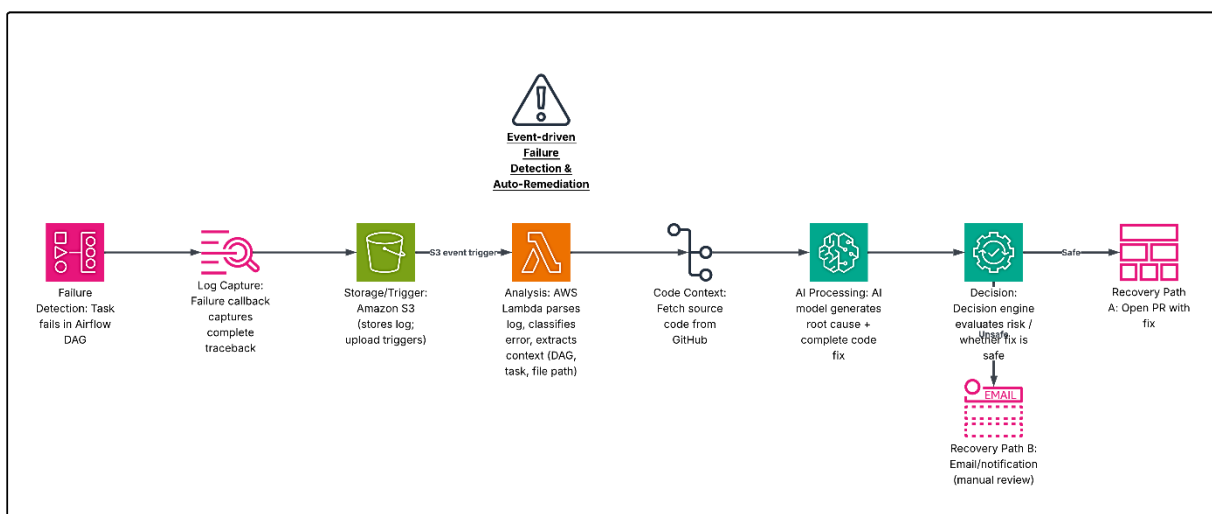


Fig 6.4 Entire System Flow

7.SETTING UP ENVIRONMENT

7. SETTING UP ENVIRONMENT

The successful implementation of the Self-Healing Astro Airflow Pipeline required setting up a robust development and execution environment. This environment includes Apache Airflow (via Astronomer Runtime), Docker-based containerization, and a MySQL database instance for testing database pipelines.

The setup process ensures that the system is reproducible, scalable, and aligned with real-world data engineering practices.

7.1 Setting up Astronomer Airflow (Astro Runtime)

Astronomer (Astro) provides a managed runtime environment for Apache Airflow, simplifying deployment and development.

Steps to Set Up Astro Airflow

1. Install Astro CLI:

The Astro CLI is used to manage Airflow projects locally.

```
npm install -g @astronomer/astro-cli
```

2. Initialize a New Project:

```
astro dev init
```

This command creates the Airflow project structure, including:

- dags/ (for pipeline definitions)
- include/ (for helper modules)
- Dockerfile
- requirements.txt

3. Start the Airflow Environment:

```
astro dev start
```

This launches Airflow locally using Docker containers, including:

- Webserver (UI)
- Scheduler
- Metadata database

4. Access Airflow UI:

<http://localhost:8080>

Default credentials:

- Username: admin
- Password: admin

This setup allows developers to create, test, and monitor DAGs in a local environment before deployment.

7.2 Custom Docker Environment Configuration

To support MySQL connectivity and additional dependencies, the default Astro Docker image was customized. The Dockerfile extends the Astronomer Runtime and installs required system-level and Python dependencies.

Key steps include:

- Switching to root user to install OS-level packages
- Installing MySQL client libraries
- Installing Python dependencies

The Dockerfile used is as follows:

- Base image: Astronomer Runtime (Airflow environment)
- Installed packages:
 - gcc
 - default-libmysqlclient-dev

These packages are required to enable Python libraries (such as MySQL connectors) to communicate with the MySQL database.

After installing system dependencies, the container switches back to the Astro user and installs Python dependencies from the requirements.txt file.

This ensures compatibility between Airflow tasks and database operations.

7.3 Setting up MySQL using Docker

To simulate a real-world database environment, a MySQL instance was deployed using Docker. A separate MySQL service was defined using Docker Compose, enabling seamless integration with the Airflow environment.

The configuration includes:

- MySQL version: 8.0
- Container name: customer_mysql
- Port mapping: 3306 (default MySQL port)
- Persistent storage using Docker volumes

Volume Configuration

A Docker volume (mysql_data) is used to persist database data across container restarts. This ensures that data is not lost when the container is stopped or recreated.

8. IMPLEMENTATION / WORKING

8. IMPLEMENTATION / WORKING

The implementation of the Self-Healing Astro Airflow Pipeline focuses on demonstrating how the system operates across multiple real-world data engineering scenarios. To validate the robustness and versatility of the system, three distinct pipelines were designed and executed.

These pipelines were specifically created as demonstration pipelines to showcase the capabilities of the system in a controlled environment. Each pipeline represents a different category of data workflow, including file-based ingestion, analytics processing, and database ETL operations.

Although these pipelines are used for demonstration purposes, the proposed system is not limited to these specific use cases. The architecture is designed to be generic and scalable, enabling it to integrate seamlessly with any real-world Airflow-based data pipeline.

These pipelines were intentionally injected with various types of errors to test the system's ability to detect, analyze, and recover from failures autonomously.

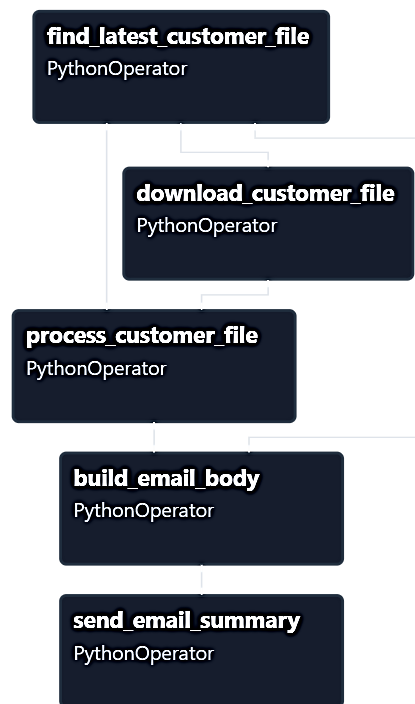
8.1 Customer Data Pipeline

Objective

The Customer Data Pipeline is designed to process customer data files stored in Amazon S3. The pipeline dynamically identifies the latest file, processes the data, and stores the results in DynamoDB. It also generates a summary email upon completion.

Workflow

1. Identify the latest file from S3
2. Download the file
3. Process customer data
4. Store processed data in DynamoDB
5. Generate and send email summary



When we trigger this pipeline and anything fails it send the logs to s3 and s3 triggers the self-healing pipeline (AWS lambda).

manual_2026-03-26T07:50:03+00:00

✖ Failed

Add a note

Clear Dag Run

Mark Dag Run as... ▼

Logical Date

Run Type

Start Date

End Date

Duration

Triggering User Name

Dag Version(s)

2026-03-26 13:20:03

▶ manual

2026-03-26 13:20:06

2026-03-26 13:20:30

00:00:23.679

Anonymous

v5

Task Instances

Asset Events

Audit Log

Code

Details

Q Search Tasks

All States ▼

Task ID

Map Index ▼

State ▼

Start Date ▼

Try Number

Operator

Duration ▼

send_email_summary

⚠ Upstream Failed

2026-03-26 13:20:28

0

PythonOperator

build_email_body

⚠ Upstream Failed

2026-03-26 13:20:27

0

PythonOperator

process_customer_file

✖ Failed

2026-03-26 13:20:22

1

PythonOperator

00:00:04.763

download_customer_file

✓ Success

2026-03-26 13:20:14

1

PythonOperator

00:00:06.966

find_latest_customer_file

✓ Success

2026-03-26 13:20:06

1

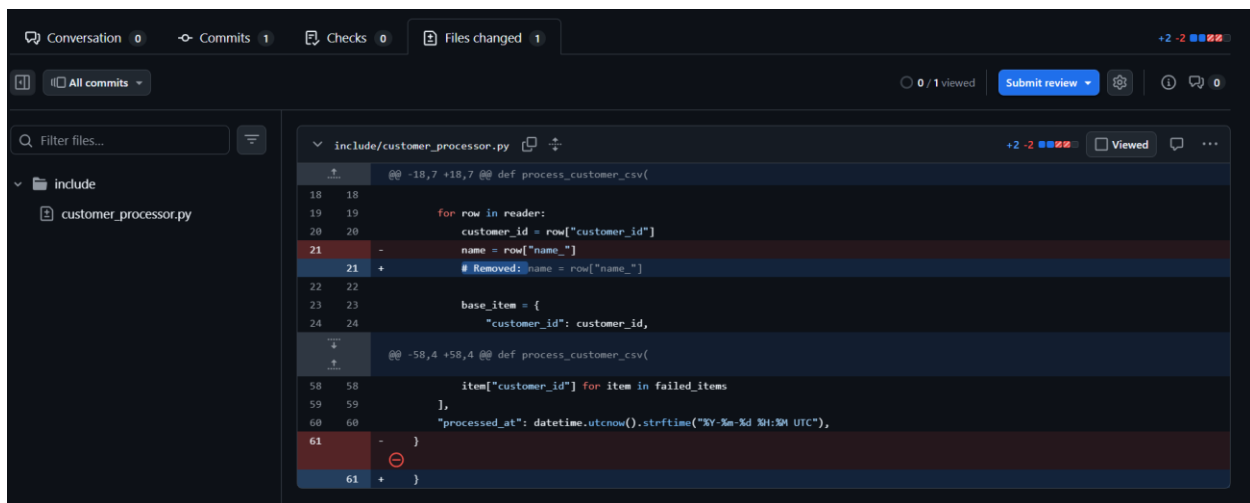
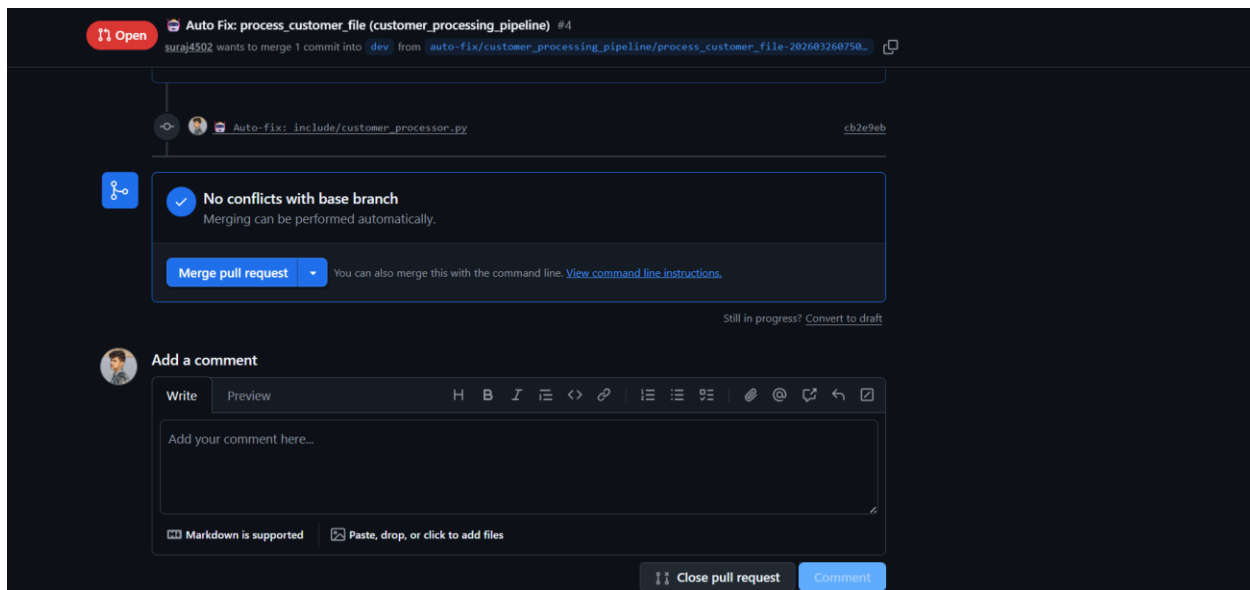
PythonOperator

00:00:07.017

So here it failed at process_customer_file task, and to fix this the self-healing pipeline analyzed the issue and created a Pull request to fix it.

This screenshot shows a GitHub Pull Request (PR) interface. At the top, the repository is 'suraj4502 / self_healing_pipeline'. The PR title is 'Auto Fix: process_customer_file (customer_processing_pipeline)' with a status of 'Ready to merge'. A comment from 'suraj4502' is visible, containing an 'Auto-Generated Fix' section. This section details the 'DAG' (customer_processing_pipeline), the 'Task' (process_customer_file), and the 'Root Cause' (a KeyError due to an incorrect column name 'name_' in a CSV file). The right sidebar shows standard PR metadata: no reviews, no assignees, no labels, and no projects.

This screenshot provides a detailed view of the 'Auto-Generated Fix' comment. It includes a 'Root Cause' section explaining the KeyError, a 'Fix' section with code changes, and a 'Classification' section with a dictionary: {'category': 'KEY_ERROR_CODE', 'confidence': 0.8, 'is_recoverable': True}. It also shows 'Confidence' (1.0) and 'Risk' (LOW). The footer notes it was 'Auto-generated by self-healing pipeline.' The right sidebar shows project settings (none yet), milestone (none), development status (successfully merging), notifications (unsubscribe button), and a lock conversation option.



So Now I can directly merge the pull request and my code will be fixed and issue will be resolved, no manual analysis and code changes required.

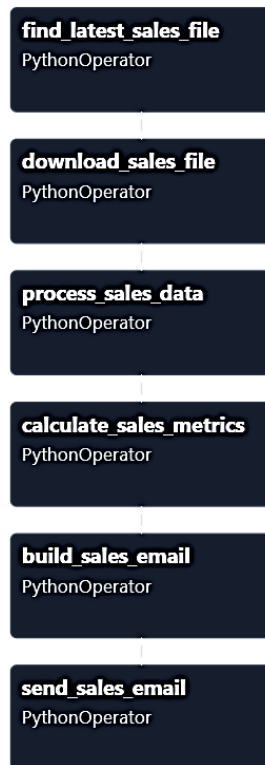
8.2 Sales Analytics Pipeline

Objective

The Sales Analytics Pipeline performs data transformation and generates business insights such as total revenue, top products, and order statistics.

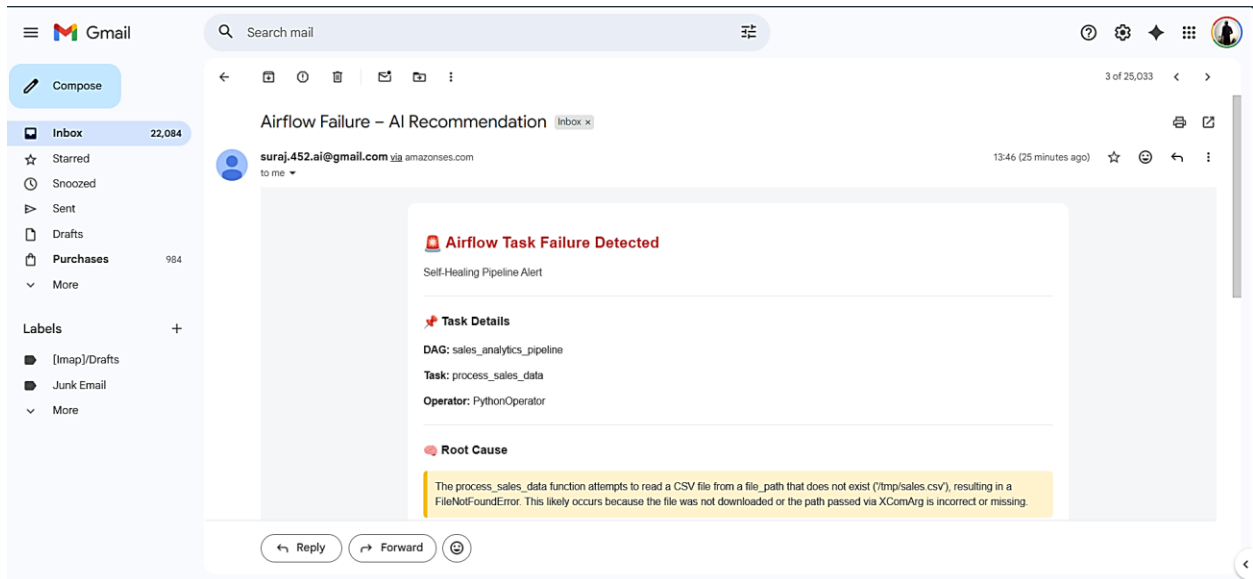
Workflow

1. Read sales data from S3
2. Clean and transform data
3. Calculate metrics
4. Generate summary report
5. Send email notification



manual_2026-03-26T08:15:55+00:00 Failed						
Add a note Clear Dag Run Mark Dag Run as...						
Logical Date	Run Type	Start Date	End Date	Duration	Triggering User Name	Dag Version
2026-03-26 13:45:55	manual	2026-03-26 13:46:02	2026-03-26 13:46:15	00:00:12.434	Anonymous	v1
Task Instances Asset Events Audit Log Code Details						
Search Tasks All States						
Task ID	Map Index	State	Start Date	Try Number	Operator	Duration
send_sales_email		Upstream Failed	2026-03-26 13:46:15	0	PythonOperator	
build_sales_email		Upstream Failed	2026-03-26 13:46:13	0	PythonOperator	
calculate_sales_metrics		Upstream Failed	2026-03-26 13:46:12	0	PythonOperator	
process_sales_data		Failed	2026-03-26 13:46:07	1	PythonOperator	00:00:05.344
download_sales_file		Success	2026-03-26 13:46:05	1	PythonOperator	00:00:01.025
find_latest_sales_file		Success	2026-03-26 13:46:03	1	PythonOperator	00:00:01.115

To resolve this issue the self-healing pipeline suggested the following changes through mail, instead of creating a pr, it intelligently decided what kind of issue is this and sent a mail about it.



✂ Suggested Fix

Add a check at the start of `process_sales_data` to verify that the file exists before attempting to read it. If the file does not exist, raise a clear exception with a descriptive error message. This will make failures more explicit and easier to debug, and prevent pandas from raising a less informative `FileNotFoundError`.

Corrected Code

```
import pandas as pd
import os

def process_sales_data(file_path, **kwargs):
    if not os.path.isfile(file_path):
        raise FileNotFoundError(f"Sales data file not found at path: {file_path}")

    df = pd.read_csv(file_path)

    # basic cleaning
    df = df.dropna()
    df["price"] = df["price"].astype(float)

    return df.to_dict(orient="records")
```

AI Confidence

0.95

⚠ Please review before applying changes.
— Self Healing Pipeline

System Behavior

- Errors classified as `DATA_SCHEMA_ERROR` or `TYPE_ERROR`
- AI generated potential fixes
- Decision engine evaluated risk
- Fix was not safe → EMAIL ONLY

This demonstrates **controlled automation**, where the system avoids risky changes.

8.3 Error Handling Workflow

The system follows a structured approach to handle failures across all pipelines.

Steps

1. Failure occurs in Airflow task
2. Failure callback captures traceback
3. Log stored in S3
4. Lambda triggered
5. Log parsed and classified
6. Context extracted
7. AI generates fix
8. Decision engine evaluates
9. Action taken (PR or Email)

8.4 AI Fix Generation and Decision Logic

The AI model plays a central role in analyzing failures and generating fixes.

Inputs to AI

- Error logs
- DAG context
- Task code
- Full source file

Outputs from AI

- Root cause
- Suggested fix
- Corrected code
- Confidence score
- Risk level

Decision Criteria

- High confidence + low risk → AUTO_PR
- Low confidence or high risk → EMAIL

8.5 Key Highlights of Implementation

- Supports multiple pipeline types
- Handles both code and data-related errors
- Uses AI for intelligent debugging
- Ensures safe automation through decision engine
- Fully event-driven and scalable

9. DEPLOYMENT

9. DEPLOYMENT

The deployment of the Self-Healing Astro Airflow Pipeline is designed to be cloud-native, scalable, and aligned with modern data engineering practices. The system integrates multiple services across orchestration, storage, serverless computing, artificial intelligence, and version control to enable a fully automated failure recovery workflow.

9.1 Deployment Architecture Overview

The system follows a distributed deployment model where each component is responsible for a specific function. Apache Airflow (via Astronomer Runtime) serves as the orchestration layer, while AWS services such as S3 and Lambda handle event-driven processing. GitHub is used for version control, and AWS SES is used for notifications.

This architecture ensures loose coupling between components, enabling independent scaling and improved fault tolerance.

9.2 Airflow Deployment using Astronomer Runtime

The Airflow environment is deployed using Astronomer Runtime, which provides a production-ready setup with Kubernetes-based execution.

Key features:

- DAG scheduling and execution
- Task monitoring and logging
- Kubernetes-based scalability
- Secure and isolated execution environment

The Airflow instance is responsible for running all pipelines and triggering the failure callback mechanism whenever a task fails.

9.3 Failure Callback Mechanism (Airflow → S3 Integration)

The failure callback mechanism plays a crucial role in enabling communication between Apache Airflow and the self-healing system. It acts as the bridge that captures failure information from Airflow and forwards it to the cloud-based processing pipeline.

In Apache Airflow, a failure callback is a user-defined function that is triggered automatically whenever a task fails. In this project, a custom failure callback (`failure_callback.py`) is implemented to capture detailed error information at the moment of failure.

Working of Failure Callback

When a task in any DAG fails, the failure callback function is executed. This function performs the following operations:

1. Captures the exception object from the Airflow context
2. Extracts the complete traceback using Python's `traceback` module
3. Collects metadata such as:
 - DAG ID
 - Task ID
 - Run ID
 - Try number
 - Exception type
 - Timestamp
1. Constructs a structured S3 key based on DAG and task hierarchy
2. Uploads:
 - The full traceback as a `.log` file
 - Metadata as a `.json` file

Role in System Architecture

This component is fundamental to the entire self-healing pipeline because:

- It enables **real-time failure capture**
- It bypasses Airflow's internal logging limitations (especially in Astro Runtime)

- It ensures logs are available in a **centralized storage system (S3)**
- It acts as the **trigger point for AWS Lambda**, which is activated via S3 events

Without this mechanism, the system would not be able to:

- Access failure logs externally
- Initiate automated recovery workflows
- Provide structured input to the AI model

The callback uses the AWS S3 Hook provided by Airflow to upload logs directly to S3. It captures the full traceback using Python's `traceback.format_exception()` function, ensuring that detailed debugging information is preserved. It also stores metadata in a structured JSON format, which is later used by downstream components such as the classifier, context extractor, and AI module.

The failure callback mechanism is a critical enabler of the self-healing system. It transforms Airflow from a passive orchestration tool into an active participant in an intelligent, event-driven recovery pipeline. By capturing failures at the source and forwarding them to the cloud, it lays the foundation for automated analysis, decision-making, and code correction.

9.4 Log Storage using Amazon S3

Amazon S3 is used as the central storage system for failure logs and metadata. When a task fails, the system stores the traceback and contextual information in S3.

Benefits of using S3:

- Highly durable and scalable storage
- Enables event-driven architecture
- Decouples Airflow from downstream processing

S3 acts as the bridge between Airflow and AWS Lambda.

9.5 Serverless Processing using AWS Lambda

AWS Lambda is deployed as the core processing engine of the system. It is triggered automatically whenever a new log file is uploaded to S3.

Responsibilities of Lambda:

- Log parsing
- Error classification
- Context extraction
- AI interaction
- Decision-making
- Triggering PR or email

Lambda ensures:

- Scalability (auto-scaling based on events)
- Cost efficiency (pay-per-use model)
- Fast execution

9.6 AI Integration (OpenAI API)

The system integrates with the OpenAI API to enable intelligent analysis and fix generation.

Deployment considerations:

- Secure API key management
- Structured prompt design
- Handling API latency and retries

The AI model processes logs and generates structured responses used by the decision engine.

9.7 GitHub Integration for Automated PR

GitHub is used as the version control platform for managing code changes.

Deployment flow:

1. Lambda fetches file from repository
2. Creates a new branch
3. Updates file with AI-generated fix
4. Creates a Pull Request

This ensures:

- Traceability
- Version control
- Human review before merge

9.8 Notification System using AWS SES

AWS SES is deployed for sending email notifications when automation is not applied. Features:

- HTML email support
- Scalable delivery
- Reliable notification system

Emails include detailed failure analysis and suggested fixes.

9.9 Security and Configuration Considerations

- API keys stored securely (environment variables)
- IAM roles used for AWS access control
- GitHub tokens managed securely
- Limited permissions for Lambda

10. LIMITATIONS

10. LIMITATIONS

While the Self-Healing Astro Airflow Pipeline demonstrates strong capabilities in automating failure detection and recovery, certain limitations must be considered.

- **Dependency on AI Accuracy**

The system relies heavily on AI-generated responses for root cause analysis and code fixes. Although the AI model performs well in many scenarios, it may occasionally generate incorrect or suboptimal solutions. This limitation necessitates the use of a decision engine to prevent unsafe or unintended changes.

- **Limited Handling of External Failures**

The system cannot fully automate recovery for failures related to external systems such as:

- Database downtime
- API failures
- Network issues

These scenarios typically require manual intervention.

- **Classification Dependency**

The effectiveness of the system depends on accurate error classification. Incorrect classification may lead to:

- Wrong decision outcomes
- Missed automation opportunities

- **API and Network Dependency**

The system depends on external services including:

- OpenAI API
- AWS services
- GitHub API

Any downtime or latency in these services can impact overall system performance.

- **Security Risks**

Automated code modification introduces potential risks such as:

- Incorrect fixes being applied
- Unauthorized or unintended changes

Although mitigated by validation rules, this remains a critical consideration.

- **Email Deliverability Issues**

Notification emails may sometimes be classified as spam due to:

- Lack of domain authentication
- Automated email patterns

- **Scalability of AI Costs**

As the number of failures increases, the cost associated with AI API usage may also increase, affecting scalability and cost-efficiency.

11. FUTURE SCOPE

11. FUTURE SCOPE

The Self-Healing Astro Airflow Pipeline lays the foundation for intelligent and autonomous data engineering systems. While the current implementation demonstrates strong capabilities, there are several opportunities to enhance and expand the system further.

- **Integration with Diverse Data Sources**

The system can be extended to support a wider range of data sources and pipeline types, making it more versatile in real-world applications. This includes:

- Streaming platforms such as Apache Kafka
- REST APIs and real-time data ingestion systems
- Cloud data warehouses such as Snowflake and BigQuery

- **Enhanced AI Accuracy and Domain Adaptation**

The performance of the system can be significantly improved by using fine-tuned or domain-specific AI models. This would enable:

- Better understanding of complex code logic
- More accurate root cause analysis
- Higher confidence in automated fixes

- **Automated Rollback and Version Control Safety**

Introducing automated rollback mechanisms can improve system reliability by reverting incorrect changes. This can include:

- Rollback of faulty pull requests
- Version comparison and validation checks
- Safe recovery strategies in case of incorrect deployments

- **Real-Time Monitoring and Visualization Dashboard**

A centralized dashboard can be developed to provide real-time insights into system performance and failures. This would allow users to monitor:

- Pipeline execution status
- Failure trends and frequency

- AI-generated decisions and actions
- Pull request history and outcomes
- **Multi-Language and Multi-Framework Support**

Currently focused on Python-based Airflow pipelines, the system can be extended to support additional languages and frameworks such as:

 - SQL-based workflows
 - Java and Scala-based data processing systems
 - Other orchestration tools beyond Airflow
- **Learning-Based Decision Engine**

The decision engine can be enhanced using machine learning techniques to improve its decision-making capabilities over time. This would enable:

 - Learning from historical fixes and outcomes
 - Adaptive confidence thresholds
 - Improved automation accuracy
- **CI/CD Pipeline Integration**

Integrating the system with CI/CD pipelines can ensure that all AI-generated fixes are automatically tested and validated before deployment. This includes:

 - Automated unit and integration testing
 - Continuous deployment workflows
 - Quality assurance checks
- **Beyond failure recovery**

The system can evolve to optimize pipeline performance. This includes:

 - Identifying inefficient DAG structures
 - Suggesting performance improvements
 - Reducing execution time and resource usage

12. CONCLUSION

12. CONCLUSION

The Self-Healing Astro Airflow Pipeline presents a novel and practical approach to addressing one of the most critical challenges in modern data engineering—pipeline failures. Traditional systems rely heavily on manual debugging and reactive monitoring, which often leads to increased downtime and operational overhead. This project introduces a shift towards intelligent, autonomous systems capable of not only detecting failures but also analyzing and resolving them with minimal human intervention.

By integrating Apache Airflow with cloud-based services such as AWS S3 and AWS Lambda, the system establishes a robust event-driven architecture for capturing and processing failures. The incorporation of Artificial Intelligence enables the system to perform root cause analysis and generate code-level fixes, significantly reducing the effort required for debugging.

A key highlight of this system is the implementation of a decision engine that evaluates AI-generated fixes based on confidence and risk levels. This ensures that automation is applied in a safe and controlled manner, preventing unintended changes while still enabling rapid recovery. The seamless integration with GitHub further enhances the system by enabling automated pull request generation, ensuring traceability and adherence to standard development workflows.

Through the implementation of multiple pipelines, including file-based ingestion, analytics processing, and database ETL workflows, the system demonstrates its versatility and adaptability across different real-world scenarios. The use of controlled failure injection further validates the system's ability to handle diverse error types and make intelligent decisions.

Moreover, the system is designed to be generic and scalable, allowing it to be integrated with any Airflow-based data pipeline. This highlights its potential for adoption in real-world production environments, where reliability and efficiency are critical.

The proposed system not only addresses current challenges but also lays the foundation for future advancements in automated data engineering and intelligent system design.

REREFERNCES

13. REREFERNCES

The following references were used during the design, development, and implementation of the Self-Healing Astro Airflow Pipeline:

Orchestration & Airflow

- Apache Airflow Documentation
<https://airflow.apache.org/docs/>
- Astronomer (Astro) Documentation
<https://docs.astronomer.io/>
- Apache Airflow GitHub Repository
<https://github.com/apache/airflow>

Cloud Services (AWS)

- AWS Lambda Documentation
<https://docs.aws.amazon.com/lambda/>
- Amazon S3 Documentation
<https://docs.aws.amazon.com/s3/>
- AWS Simple Email Service (SES) Documentation
<https://docs.aws.amazon.com/ses/>
- AWS IAM (Identity and Access Management) Documentation
<https://docs.aws.amazon.com/iam/>

Artificial Intelligence

- OpenAI API Documentation
<https://platform.openai.com/docs/>

Version Control & APIs

- GitHub REST API Documentation
<https://docs.github.com/en/rest>

- GitHub Actions & Workflows Documentation
<https://docs.github.com/en/actions>

Containerization & Databases

- Docker Documentation
<https://docs.docker.com/>
- Docker Compose Documentation
<https://docs.docker.com/compose/>
- MySQL Documentation
<https://dev.mysql.com/doc/>

Conceptual References

- Event-Driven Architecture Concepts
<https://aws.amazon.com/event-driven-architecture/>
- ETL (Extract, Transform, Load) Concepts
<https://www.ibm.com/topics/etl>